CIS 4910

Week 6: From Requirements to Architecture

Turning your SRS into a Detailed Design Document: decomposition, coupling and cohesion, and the architectural view you commit to.

Professor Scot Hollingsworth

scoth@usf.edu

Office hours by appointment only



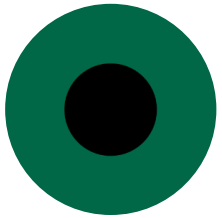
SESSION ROADMAP, PART 1 OF 2

Agenda for Week 6

01	The Fifteen Week Reality	5 min
02	From SRS to HLA to DDD	30 min
03	High Level Architecture, Deep Dive	25 min
04	Coupling and Cohesion	30 min
05	Architectural Views and Styles	20 min
06	Team Workshop	15 min
07	Takeaways, Looking Ahead	10 min

BY THE END OF TODAY

Learning Objectives



- Name every section a Detailed Design Document must contain
- Trace a requirement to the module and pattern that satisfies it
- Decompose a system using a named strategy, not gut feeling
- Diagnose coupling and cohesion problems in your own design
- Commit to an architectural view and style, and defend it

01

The Fifteen-Week Reality

A thirty second reminder before we go deep

Speed Without Design Is Just Fast Failure

- AI lets a small team generate code faster than ever before
- Faster code generation does not mean faster correct systems
- Every hour skipped in design gets repaid, with interest, later, often at a very high rate of return.
- Today gives you the required content your DDD and your AI prompts both need

15

weeks total

Weeks 6 and 7 are your last checkpoint before the mid term. Detailed Design!

02

From SRS to HLA to DDD

What each document must actually contain

What Your SRS Already Gave You

- A numbered list of functional and nonfunctional requirements, written at specification level detail
- A one-page HLA sketch: boxes for subsystems, arrows for data flow
- A rough, early sense of components, not yet a real architecture
- Constraints: platform, performance targets, legal obligations
- Legal: What you can and can not do in a project

None of this disappears. It is the raw material the DDD builds on.

What a Detailed Design Document Must Contain

1. Architecture overview (HLA), enhanced after the SRS presentation to reflect current reality
2. Decomposition, module specs
3. Interface specifications
4. Data design and schema
5. Design patterns used, and why
6. Quality attribute design
7. Build versus buy analysis
8. Traceability to requirements/specifications
9. Risks, assumptions, open issues

A Detailed Design Document SHOULD **NOT** Contain the following:

1. A restatement of requirements / specifications
2. Management, Risk or Methodology tracking plan
3. Test Plan, that goes in the updated SRS for week #12 deliverable in MVP1. Sometimes a test plan is its own artifact, standalone, but for this class it becomes part of the SRS, and specific tests should reference specific requirements and specs.

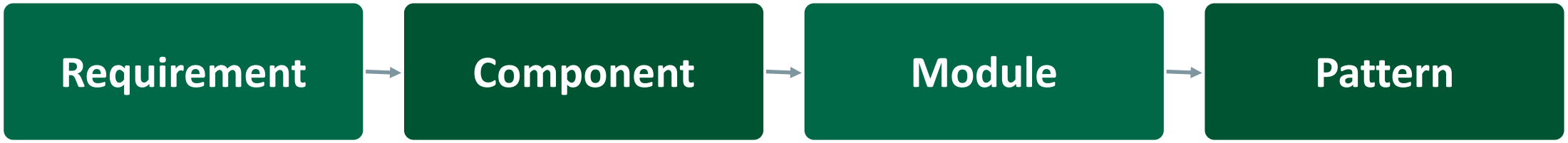
Merging or renaming sections to fit your project is fine. Missing one is not.

The Standard Behind Your Views

- 42010 is the international standard for how an architecture description should be organized
- Stakeholders have concerns; concerns are addressed by viewpoints; viewpoints produce views
- The Four Plus One model later in this section is one well known viewpoint framework built on this idea
- Naming the framework in your DDD tells a reviewer your views were chosen on purpose, not assembled after the fact
- Represent each view with a recognized notation: UML diagrams, or C4 model levels, Context, Container, Component, Code

The Traceability Chain: Requirement to Component

Every requirement should point at the exact design element that satisfies it.



- REQ-014: user can reset a forgotten password
- Component: Authentication Service
- Module: PasswordResetHandler
- Pattern: Strategy, reset by email or by SMS

If you cannot draw this chain for a requirement, your design has a gap.

From a One-Page Sketch to a Real Architecture

Your SRS sketch had	Your DDD architecture adds (MUST HAVE)
Boxes with names	One responsibility sentence per box
Arrows between boxes	Direction, protocol, and data format
An implied data store	A schema: entities and relationships
No stated qualities	Targets: response time, uptime, users

Interface Specification: What Good Looks Like

Field	Example: resetPassword
Signature	resetPassword(userId, method)
Inputs	userId: string, method: EMAIL or SMS
Outputs	Result: SUCCESS, or an error code
Pre and postconditions	userId is active; a reset token is issued
Error handling	Unknown userId returns NOT_FOUND

Quality Attributes and Cross Cutting Concerns

1. **Security:** authentication, authorization, and data protection built in, not bolted on
2. **Observability:** logging, monitoring, and tracing so a failure can be found, not just noticed
3. **Fault tolerance:** what happens when a dependency times out or a service goes down
4. **Configuration management:** how settings change across environments without a code change

1. **Performance versus scalability:** fast for one user is not the same as fast for one thousand
2. **Consistency versus availability:** the tradeoff your data design already makes, whether you named it or not
3. **Quantitative targets:** expected load, latency budget, and capacity plan, stated as numbers
4. **Validation:** how you will show the design holds up, prototyping, simulation, or benchmarking

Every item here needs at least a sentence in your DDD.

DDD Checklist, Mapped to the Rubric

Rubric criterion	DDD sections that answer it
Architecture and technical content	Overview, decomposition, interfaces, patterns
Coordinated, professional presentation	Traceability, quality attributes, risks
Ready to move to implementation	Data design, build versus buy, module specs

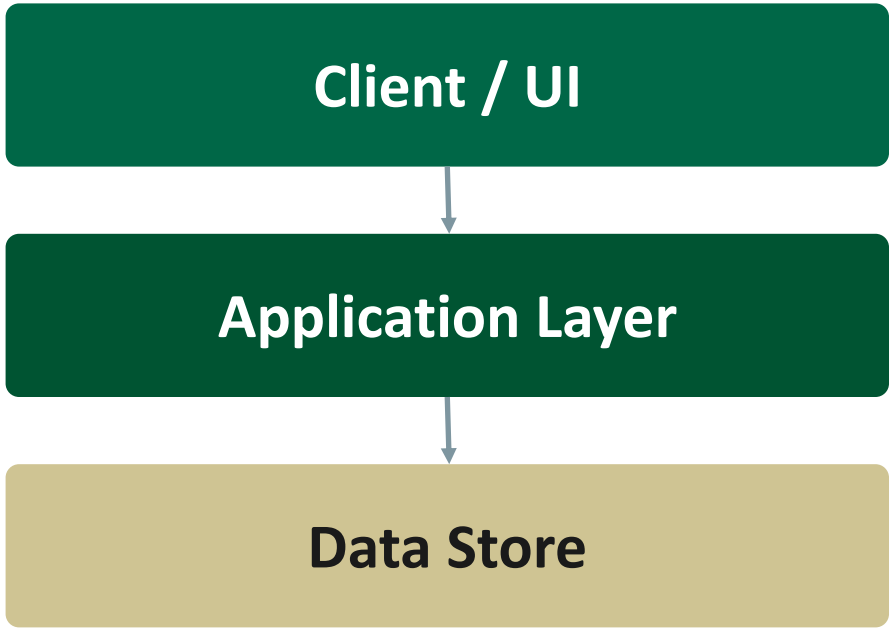
03

High Level Architecture, Deep Dive

From a sketch to a defensible structure

HLA: Purpose and the One Page View

- One slide, one picture: the whole system at a glance
- Shows major subsystems and connections, not detail
- A stranger should understand your system in under a minute



Decomposition Strategies: Three Named Approaches

Most capstone projects blend all three below.

Functional

Split by what it does

Login, Search, Checkout

Object Oriented

Split by the things it manages

User, Order, Product

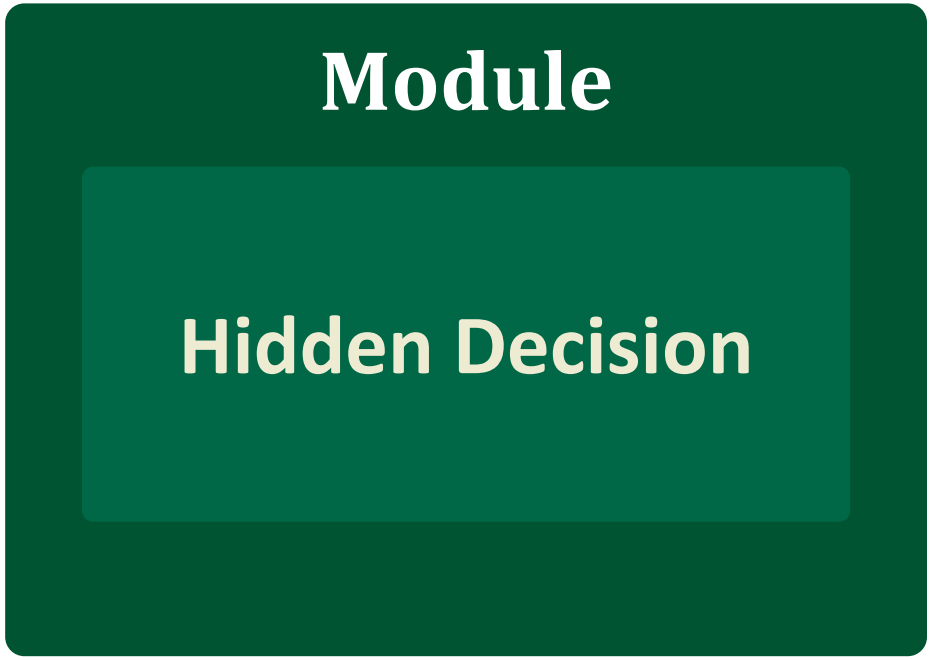
Data Driven

Split by how data flows

Ingest, Validate, Store

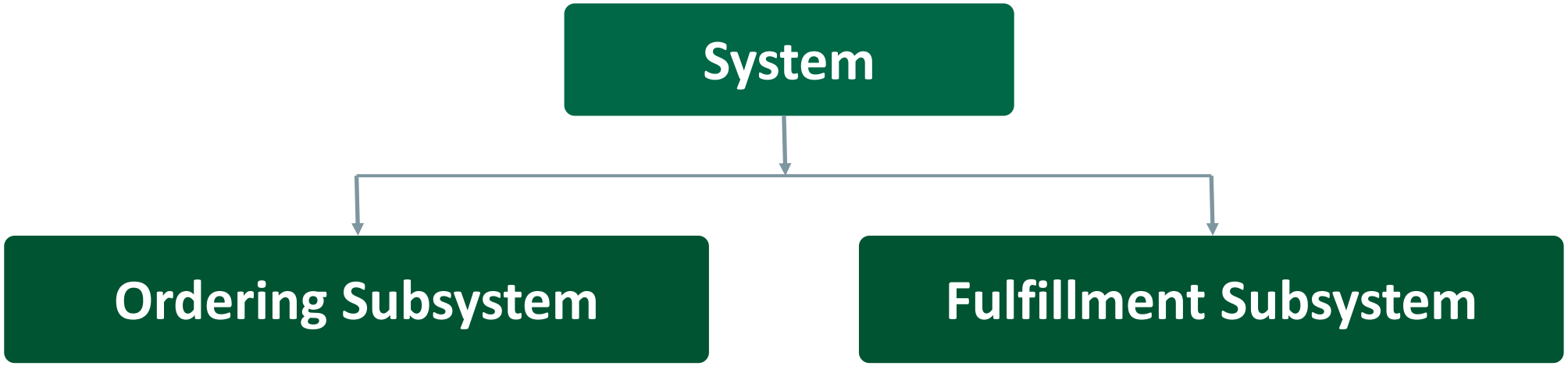
Module Granularity and Information Hiding

- Stop decomposing when a module has one reason to change
- Each module hides a design decision likely to change
- Other modules see only the interface, never the decision inside



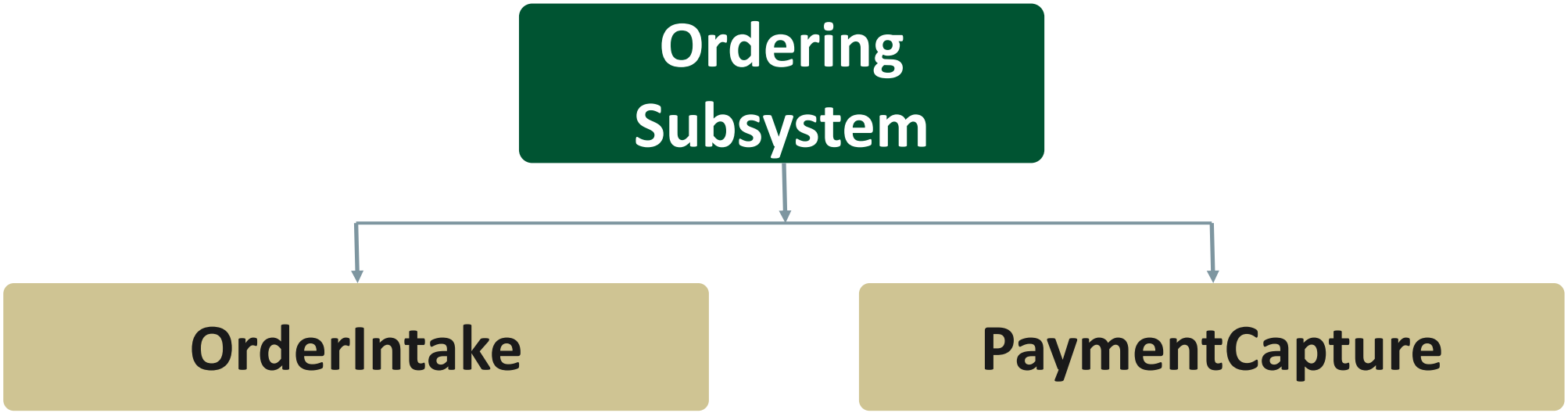
Interface (visible)

Decomposition Worked Example



Next: zooming into the Ordering Subsystem's own modules.

Zooming Into the Ordering Subsystem



Stop here: each module has one clear owner on your team.

04

Coupling and Cohesion

The two properties that decide your SPRINT velocity

REMEMBER: for this class a sprint is 1 week; in typical industry practice, sprints run 2 weeks, or about 10 working days

Why Coupling and Cohesion Decide Your Velocity

- **Coupling:** how tangled your modules are with each other
- **Cohesion:** how focused a single module is on one job
- Both run on a scale, worst to best, not a yes or no
- A team of three or four cannot parallelize through tight coupling
- Every AI generated component inherits the coupling you designed for it

Content Coupling

AVOID

One module directly reaches into and modifies another module's internal data. Changing either module silently breaks the other.

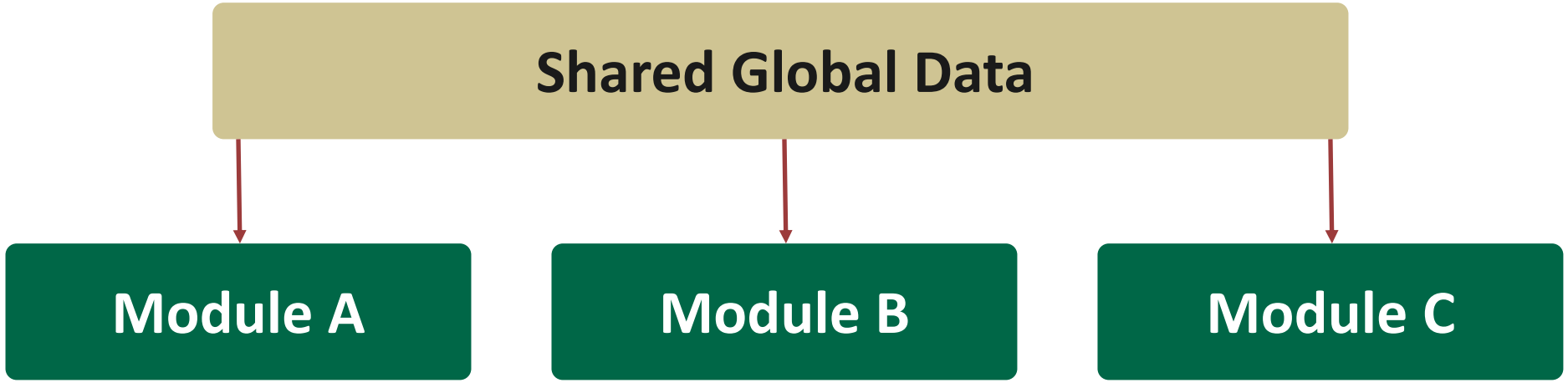


Module A reaches directly into Module B's private data

Common Coupling

AVOID

Multiple modules read and write the same shared global data. A change anywhere can silently affect every module touching it.



Stamp Coupling

OK

One module passes an entire data structure to another, which only needs a few of its fields. Workable, but the receiver now depends on the whole shape.

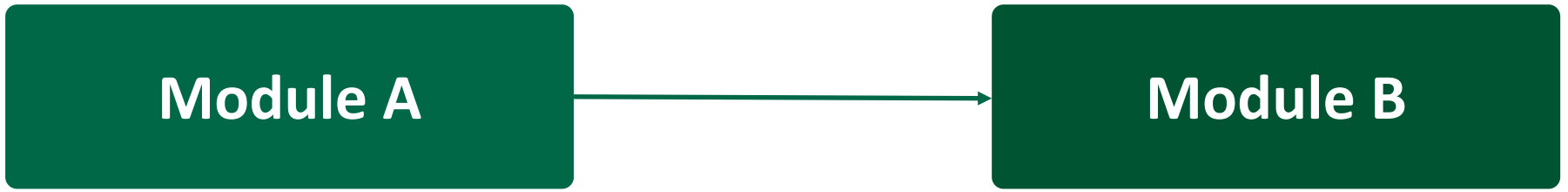


Whole record passed, though only a few fields are used

Data Coupling

TARGET

Modules pass only the specific primitive values a call needs. Each module can change internally with no effect on the other.



Only userId and method are passed between them

Coincidental Cohesion

AVOID

Unrelated tasks are bundled into one module for convenience. A grab bag utility class is the classic example.



Temporal Cohesion

WEAK

Tasks are grouped only because they happen at the same time, such as a startup routine. They share timing, not purpose.



Communicational Cohesion

GOOD

Tasks operate on the same set of data, such as reading, validating, and formatting one order record. Common and workable in practice.



Functional Cohesion

TARGET

Every part of the module works together toward exactly one well defined task. This is the goal for every module you design.

validatePassword()

SELF CHECK

Diagnosing Your Own Design

1. Can module A change its data structure without touching module B?
2. Does every function in this module serve the same single purpose?
3. Could two teammates edit two modules today with no merge conflict?
4. If not yet, that is exactly what the workshop today is for

05

Architectural Views and Styles

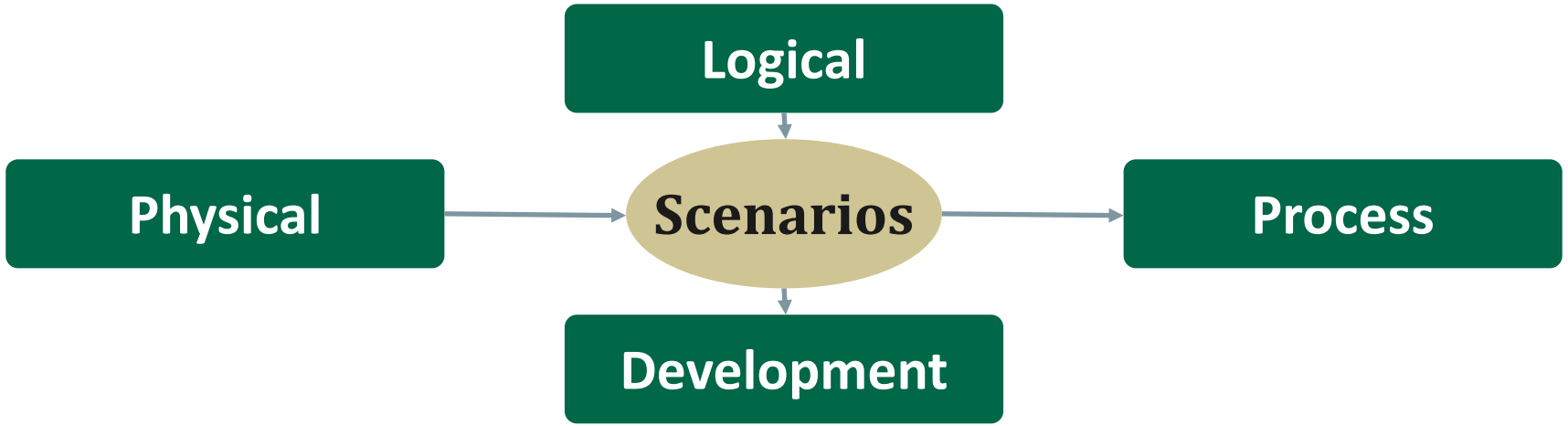
State your choice, then defend it

Choosing Your Architectural Style

Style	Best fit	Watch out for
Layered	Simple web or desktop apps	Layers as formality only
Client / Server	Front end, back end split	Overloading the server
Microservices lite	Independent modules	Too much overhead
Event driven	Reacting to data streams	Harder to debug

The Four Plus One View Model

State which views your DDD documents. Scenarios sit at the center because every other view must satisfy them.



- Logical: the object or class model
- Process: concurrency and runtime behavior
- Development: source code organization
- Physical: deployment across hardware

Behavioral View: Sequence Diagrams

Shows how components exchange messages over time for one key workflow.

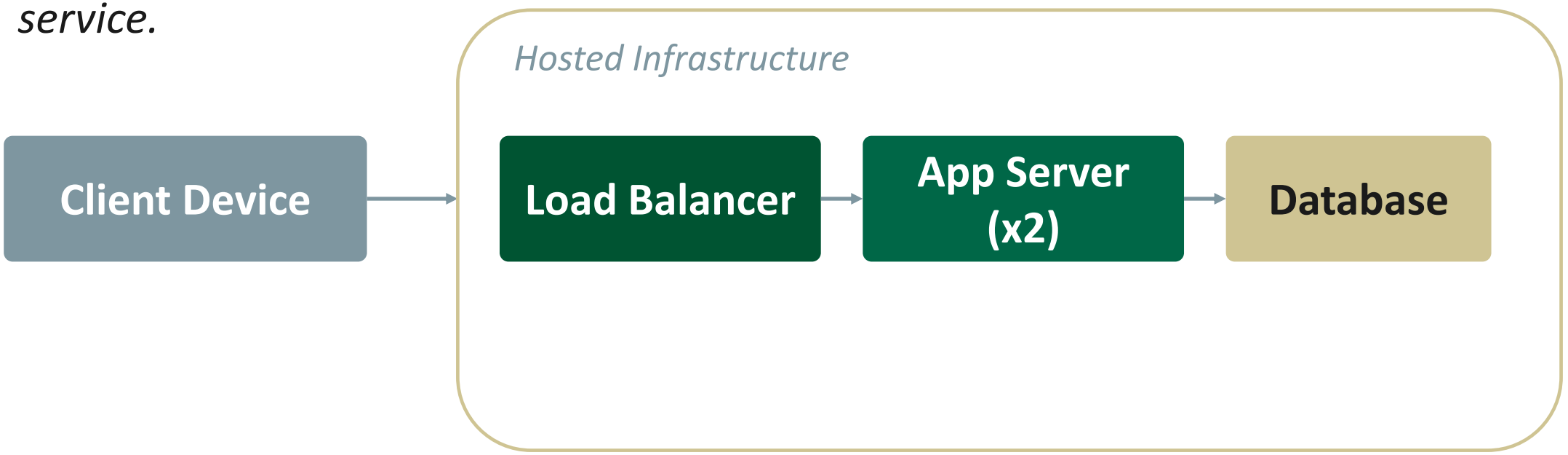


1. User sends a login request to the UI
2. UI asks the Auth Service to validate the credentials
3. Auth Service queries the Database for the user record
4. Database returns the result back to the Auth Service

Use when: a workflow's timing and message order matter as much as which components are involved, such as authentication, checkout, or an approval flow.

Physical View: Deployment Diagrams

Shows where each component actually runs: which machine, container, or managed service.



Use when: your DDD must show infrastructure, redundancy, and where a request actually travels, not just which module handles it.

Team Workshop



- Draw one requirement's full traceability chain to a module
- Find a pair of modules and name their coupling type
- Find a module, name its cohesion type; propose a split if weak
- State your architectural style and view choices, and defend them

Mapping Today Back to Your DDD

- Traceability chains become your traceability section
- Decomposition strategy and diagrams become your architecture overview
- Coupling and cohesion findings go in your module specifications
- Your view and style get named, with one sentence of justification each

Key Takeaways

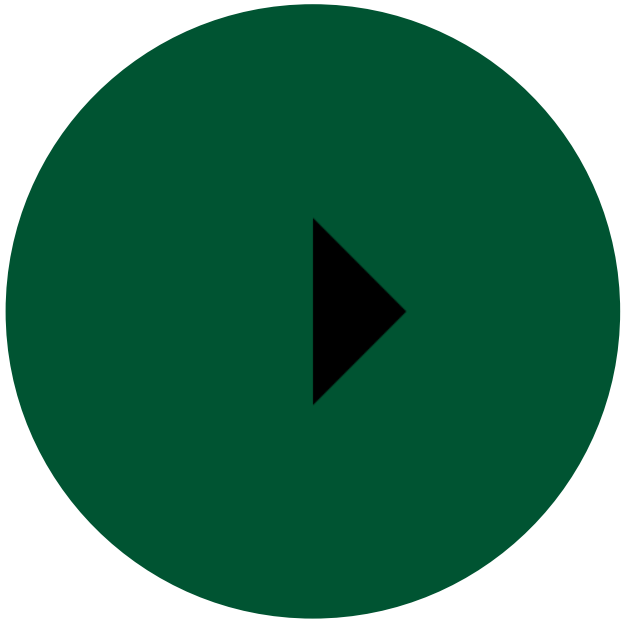


- A DDD has required sections; missing one is a gap a reviewer finds
- Every requirement should trace to a component, module, and pattern
- Decompose with a named strategy; stop at one owner per module (For this class)
- **Aim for data coupling and functional cohesion**

NEXT SESSION

Looking Ahead to Week 7

- Design patterns: a full tour across all three GoF families
- AI in design: real prompts, ADR writing, and verification
- AI pitfalls and guardrails, expanded with real failure modes
- Methodology: Agile, AI SE, TDD, and FDD combined



Questions?

CIS 4910, Week 6

Professor Scot Hollingsworth

scoth@usf.edu